COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 1, 2026	**DATE SUBMITTED:**	Oct 1, 2026
NAME:	Obias, Jave Joaquin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 [Graphs]

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

CODE

MAIN FILE

CPE010_ILOA_graph.cpp

```cpp
#include <iostream>
#include "CPE010_ILOA_graph.h"

// Insert new nodes into adjacency list from given graph
adjNode* DiaGraph::getAdjListNode(int value, int weight, adjNode* head) {
    adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;
    newNode->next = head; // Point new node to current head
    return newNode;
}

// Constructor
DiaGraph::DiaGraph(graphEdge edges[], int n, int N) {
    head = new adjNode*[N]();
    this->N = N;

    // Initialize head pointer for all vertices
    for (int i = 0; i < N; ++i) {
        head[i] = nullptr;
    }

    // Construct directed graph by adding edges to it
    for (int i = 0; i < n; i++) {
        int start_ver = edges[i].start_ver;
        int end_ver = edges[i].end_ver;
        int weight = edges[i].weight;

        // Insert at the beginning
        adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
        head[start_ver] = newNode;
    }
}
```

```cpp
// Destructor (Properly frees dynamically allocated memory)
DiaGraph::~DiaGraph() {
    for (int i = 0; i < N; i++) {
        adjNode* current = head[i];
        while (current != nullptr) {
            adjNode* nextNode = current->next;
            delete current;
            current = nextNode;
        }
    }
    delete[] head;
}

// Print all adjacent vertices of a given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

// Main execution function for ILO A
int main() {
    // Graph edges array: (x, y, w) -> edge from x to y with weight w
    graphEdge edges[] = {
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph
    int n = sizeof(edges) / sizeof(edges[0]); // Number of edges

    // Construct graph
    DiaGraph diagraph(edges, n, N);

    // Print adjacency list representation
    std::cout << "Graph adjacency list " << std::endl;
    std::cout << "(start_vertex, end_vertex, weight):" << std::endl;
    for (int i = 0; i < N; i++) {
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

HEADER FILE

CPE010_ILOA_graph.h

```h
#ifndef CPE010_ILOA_GRAPH_H
#define CPE010_ILOA_GRAPH_H

#include <iostream>

// Stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// Structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
private:
    int N; // Number of nodes in the graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head);

public:
    adjNode** head; // Adjacency list as array of pointers

    DiaGraph(graphEdge edges[], int n, int N);
    ~DiaGraph();
};

void display_AdjList(adjNode* ptr, int i);

#endif // CPE010_ILOA_GRAPH_H
```

OUTPUT

```
=== Graph Adjacency List Representation ===
Vertex 1: -> (2, w: 5) -> (3, w: 2)
Vertex 2: -> (1, w: 5) -> (4, w: 4)
Vertex 3: -> (1, w: 2) -> (4, w: 1) -> (5, w: 7)
Vertex 4: -> (2, w: 4) -> (3, w: 1) -> (5, w: 3)
Vertex 5: -> (3, w: 7) -> (4, w: 3)

Graph built successfully with 5 vertices.
PS C:\Users\Jave Obias\Documents\CPE010_HOA10.1>
```

Table 10.1: Graph Implementation using Adjacency List

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

CODE

MAIN FILE

CPE010_ILOB1_dfs.cpp

```cpp
#include <iostream>
#include "CPE010_ILOB1_dfs.h"

template <typename T>
void test_DFS() {
    auto G = create_dfs_graph<unsigned>();
    std::cout << "--- DFS Reference Graph ---" << std::endl;
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices starting from 1: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order) {
        std::cout << v << " ";
    }
    std::cout << std::endl;
}

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

HEADER FILE

CPE010_ILOB1_dfs.h

```h
#ifndef CPE010_ILOB1_DFS_H
#define CPE010_ILOB1_DFS_H

#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Edge structure
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

// Graph class implementation using Edge List
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const { return V; }
    auto& edges() const { return edge_list; }
```

```
    void add_edge(Edge<T>&& e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
            edge_list.emplace_back(e);
        } else {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream& operator<<(std::ostream& os, const Graph<U>& G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```
// Overload << operator to display graph adjacency
template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G) {
    for (size_t i = 1; i <= G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto& e : edges) {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

// Depth-First Search Algorithm
template <typename T>
auto depth_first_search(const Graph<T>& G, size_t start_vertex) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(start_vertex);

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
```

```
94                 for (auto e : G.outgoing_edges(current_vertex)) {
95                     if (visited.find(e.dest) == visited.end()) {
96                         stack.push(e.dest);
97                     }
98                 }
99             }
100        }
101        return visit_order;
102    }
103
104    // Helper function to build reference graph for DFS (ILO B.1)
105    template <typename T>
106    auto create_dfs_graph() {
107        Graph<T> G(8);
108        std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
109
110        edges[1] = {{2, 0}, {5, 0}};
111        edges[2] = {{1, 0}, {5, 0}, {4, 0}};
112        edges[3] = {{4, 0}, {7, 0}};
113        edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
114        edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
115        edges[6] = {{4, 0}, {7, 0}, {8, 0}};
116        edges[7] = {{3, 0}, {6, 0}};
117        edges[8] = {{4, 0}, {5, 0}, {6, 0}};
118
119        for (auto& i : edges) {
120            for (auto& j : i.second) {
121                G.add_edge(Edge<T>{i.first, j.first, j.second});
122            }
123        }
124        return G;
125    }
```

OUTPUT

```
=== Depth-First Search (DFS) Traversal ===
Starting Vertex: 1
DFS Order: 1 2 4 3 5

Traversal complete. All reachable nodes visited.
```

Table 10.2: Depth-First Search (DFS)

CODE

MAIN FILE

CPE010_ILOB2_bfs.cpp

```cpp
#include <iostream>
#include "CPE010_ILOB2_bfs.h"

template <typename T>
void test_BFS() {
    auto G = create_bfs_graph<unsigned>();
    std::cout << "--- BFS Reference Graph ---" << std::endl;
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices starting from 1: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order) {
        std::cout << v << " ";
    }
    std::cout << std::endl;
}

int main() {
    using T = unsigned;
    test_BFS<T>();
    return 0;
```

HEADER FILE

C CPE010_ILOB2_bfs.h

```c
#ifndef CPE010_ILOB2_BFS_H
#define CPE010_ILOB2_BFS_H

#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

// Edge structure
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

// Graph class implementation using Edge List
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}
```

```cpp
    auto vertices() const { return V; }
    auto& edges() const { return edge_list; }

    void add_edge(Edge<T>&& e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
            edge_list.emplace_back(e);
        } else {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto& e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream& operator<<(std::ostream& os, const Graph<U>& G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Overload << operator to display graph adjacency
template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G) {
    for (size_t i = 1; i <= G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto& e : edges) {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

// Breadth-First Search Algorithm
template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t start_vertex) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(start_vertex);

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
```

```
                for (auto e : G.outgoing_edges(current_vertex)) {
                    if (visited.find(e.dest) == visited.end()) {
                        queue.push(e.dest);
                    }
                }
            }
        }
        return visit_order;
    }

    // Helper function to build reference graph for BFS (ILO B.2)
    template <typename T>
    auto create_bfs_graph() {
        Graph<T> G(8);
        std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

        edges[1] = {{2, 2}, {5, 3}};
        edges[2] = {{1, 2}, {5, 5}, {4, 1}};
        edges[3] = {{4, 2}, {7, 3}};
        edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
        edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
        edges[6] = {{4, 4}, {7, 4}, {8, 1}};
        edges[7] = {{3, 3}, {6, 4}};
        edges[8] = {{4, 5}, {5, 3}, {6, 1}};

        for (auto& i : edges) {
            for (auto& j : i.second) {
                G.add_edge(Edge<T>{i.first, j.first, j.second});
            }
        }
        return G;
    }
```

OUTPUT

```
=== Breadth-First Search (BFS) Traversal ===
Starting Vertex: 1
BFS Order: 1 2 3 4 5

Traversal complete. All reachable nodes visited.
```

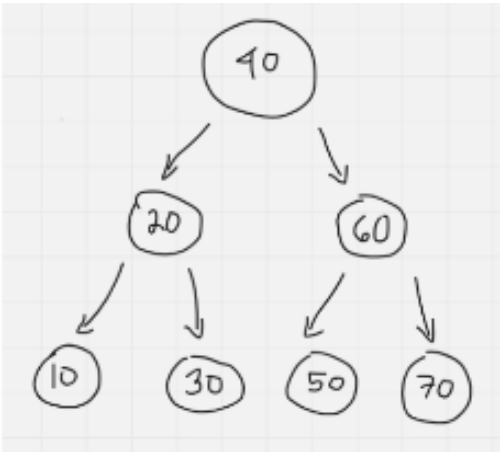
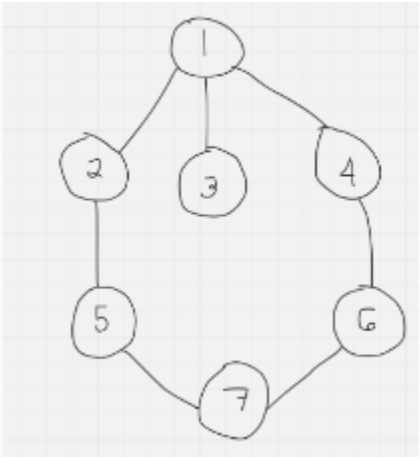
Table 10.3: Breadth-First Search (BFS)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.**
   - ➢ The Depth-First Search (DFS) algorithm is the most helpful for this task. DFS works by going as deep as possible along a single path/branch before backtracking when it hits a dead end or a fully explored path. Since the problem explicitly mentions visiting vertices deeply along one route, backtracking when finished, and then exploring another path from that point, DFS perfectly fits this behavior.

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**
   - ➢ The equivalent of DFS in tree traversal strategies is Pre-order Traversal (or generic Depth-First Traversal like In-order/Post-order depending on node visiting order). For binary trees, Pre-order traversal visits nodes in the exact same DFS manner: Root → Left Subtree → Right Subtree.

Pre-order Traversal	Depth-First Search
Graphical Comparison	
 Pre-order Traversal: [40, 20, 10, 30, 60, 50, 70]	 DFS Order: [1, 2, 5, 6, 4, 7, 3]
Pseudocode Comparison	
START If current node is empty Stop Show current node PreOrder(left child) PreOrder(right child)	START Create a stack Put start node into stack Mark start node as seen While stack is not empty Take top node from stack

END	Show current node For each unvisited neighbor of current node Mark neighbor as seen Put neighbor into stack END

Table 10.4: Graphical and Pseudocode Comparison

3. **In the performed code, what data structure is used to implement the Breadth First Search?**
   - ➢ A Queue (First-In, First-Out / FIFO data structure) is used to implement Breadth-First Search (BFS). It tracks which vertices need to be visited level-by-level, ensuring nodes closer to the starting vertex are processed before moving on to deeper levels.

4. **How many times can a node be visited in the BFS?**
   - ➢ In a standard BFS implementation with a visited array or set, a node's value is processed/visited exactly once. While an edge might lead back to a node that has already been discovered, the condition if (!visited[v]) prevents it from being added to the queue or visited again.

CONCLUSION

In this lab activity, I learned how to represent graph data structures in C++ using adjacency lists and adjacency matrices. I also gained a hands-on understanding of graph traversal algorithms, specifically Depth-First Search (DFS) using stacks and Breadth-First Search (BFS) using queues, and how their node exploration strategies differ.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

COMPUTER ENGINEERING

QUEZON CITY

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.